Detección de personas y armas en videovigilancia mediante YOLOv5: análisis de objetos pequeños, entrenamiento Multi-Scale y generación de alertas

Jose Alvarado, Dina Huaman, Jose Laymito, Christopher Nuñez, Frank Huaman
Maestría en Inteligencia Artificial, Facultad de Ingeniería Industrial y de Sistemas
Universidad Nacional de Ingeniería, Lima, Perú
jalvaradom@uni.pe, dina.huaman.h@uni.pe,
jose.laymito.q@uni.pe, c.nunez.v@uni.pe,
frank.huaman.c@uni.pe

***Abstract*—Este trabajo estudia la detección automática de personas y armas en imágenes de videovigilancia mediante YOLOv5s, con énfasis en la dificultad que representan las armas de pequeño tamaño. Se construyó una partición reproducible del conjunto YouTube-GDD sin solapamiento de videos entre entrenamiento, validación y prueba. Sobre una línea base B1 se evaluaron dos modificaciones controladas: M1 aumentó la resolución de entrada de 640 a 1280 y M2 incorporó entrenamiento Multi-Scale a 640. M1 redujo el mAP@0.5:0.95 de validación de 0.669 a 0.524 y elevó 76% el tiempo de entrenamiento; el análisis por objeto mostró que los errores persistentes se concentran en armas pequeñas, periféricas y en escenas con varias armas. En la corrida inicial, M2 obtuvo el mejor fitness de validación (0.698 frente a 0.688) y fue seleccionado y congelado antes de acceder a Test, donde alcanzó mAP@0.5 de 0.845 y mAP@0.5:0.95 de 0.659 (0.506 para *gun*). Sin embargo, al repetir B1 y M2 hasta reunir cuatro entrenamientos de cada uno, la diferencia desapareció (mAP@0.5:0.95 de 0.679 $\pm$ 0.007 frente a 0.674 $\pm$ 0.005; $p = 0.31$) y el recall de armas pequeñas se mantuvo por debajo de 0.20 en ambos. Como demostración práctica, el detector se integró en un prototipo en Streamlit con un motor temporal que exige persistencia de las detecciones antes de emitir alertas para revisión humana.**

***Index Terms*—YOLOv5, detección de armas, videovigilancia, objetos pequeños, Multi-Scale, aprendizaje profundo, Streamlit.**

I. Introducción

Los sistemas de videovigilancia producen flujos continuos de información visual cuya supervisión exclusiva por operadores humanos puede resultar difícil de sostener. Los modelos de visión por computadora pueden funcionar como herramientas de apoyo para localizar eventos visuales de interés y priorizar su revisión. La detección de armas presenta una dificultad adicional: el objeto puede ocupar una fracción muy reducida de la escena, aparecer parcialmente ocluido, desplazado hacia la periferia, afectado por movimiento o acompañado por objetos visualmente similares [1], [2].

El problema se formula como detección multiclase de las categorías *person* y *gun*. A diferencia de una clasificación binaria, el detector debe determinar qué objeto aparece y dónde se encuentra mediante cajas delimitadoras. El objetivo no es identificar individuos ni inferir intenciones, sino generar evidencia visual que pueda apoyar una revisión humana. Esta delimitación es relevante por el impacto potencial de falsos positivos y falsos negativos en una aplicación de alerta.

La pregunta experimental central es si una estrategia de entrenamiento Multi-Scale puede mejorar la detección de armas pequeñas respecto de una línea base YOLOv5s, manteniendo condiciones controladas y un costo computacional razonable. La investigación sigue una secuencia basada en evidencia: establecer B1, diagnosticar errores, evaluar M1, profundizar el análisis y formular M2 a partir de los hallazgos.

Las contribuciones del trabajo son:

- Preparación de una partición reproducible de YouTube-GDD sin solapamiento de videos entre Train, Validation y Test, documentando las correcciones realizadas.
- Comparación controlada de tres configuraciones YOLOv5s en las que se modificó una variable experimental principal por etapa.
- Caracterización de los errores de la clase *gun* según tamaño, posición y densidad de escena, complementada con diagnóstico de anchors y representatividad de Train.
- Selección y congelamiento del mejor modelo antes de acceder a Test, seguido de evaluación independiente.
- Estimación de la variabilidad entre entrenamientos con cuatro corridas de B1 y de M2, que muestra que la ventaja de M2 observada con una sola semilla no se sostiene.

El resto del artículo se organiza así: la Sección II presenta los trabajos relacionados; la Sección III describe los datos, la arquitectura y el diseño experimental; la Sección IV presenta los resultados; la Sección V desarrolla la evaluación independiente y la propuesta práctica; la Sección VI discute los hallazgos y limitaciones; y la Sección VII presenta las conclusiones.

II. Trabajos Relacionados

Los detectores de una etapa de la familia YOLO formulan la detección como una regresión directa de cajas y clases sobre una cuadrícula [3]. YOLOv2 introdujo el entrenamiento multiescala, que cambia la resolución de entrada cada pocos lotes para que una misma red funcione a distintas resoluciones [4]; M2 aplica esta idea en YOLOv5 [5], cuyo backbone se basa en CSPNet [6] y cuyo cuello agrega características de

varios niveles con PANet [7]. Los pesos iniciales provienen de COCO [8], y los datos, de YouTube-GDD [9], que reúne fotogramas de videos públicos con anotaciones de personas y armas.

La detección de armas en videovigilancia ha sido abordada mediante detectores de objetos y estrategias destinadas a reducir falsas alarmas. Olmos, Tabik y Herrera [1] desarrollaron un sistema de detección automática de pistolas y generación de alarmas en video, prestando especial atención a la reducción de falsos positivos. Este antecedente muestra que no basta con reconocer el objeto: también importa el comportamiento temporal y la frecuencia de falsas alarmas. Salazar González *et al.* [2] estudiaron la detección de armas en CCTV real mediante Faster R-CNN con FPN y ResNet-50, y evidenciaron degradaciones asociadas a distancia, movimiento, tamaño aparente y oclusión.

Otra línea incorpora información contextual de las personas presentes en la escena. Velasco-Mata *et al.* [10] integraron información de pose humana a un detector basado en YOLOv3. Berardini *et al.* [11] propusieron un esquema de dos etapas en el que una CNN localiza personas y otra busca armas dentro de las regiones correspondientes. Estos antecedentes motivan una futura implementación persona–arma; sin embargo, dicha lógica contextual no forma parte de las comparaciones B1, M1 y M2 de este estudio.

Una tercera línea analiza condiciones visuales adversas y técnicas para mejorar la representación de objetos pequeños. Corral-Sanz *et al.* [12] estudiaron la influencia de imágenes artificialmente distorsionadas sobre el desempeño de detectores de armas. Berardini *et al.* [13] exploraron la superresolución como tarea auxiliar para mejorar la detección en videovigilancia. En este trabajo, la superresolución no se implementó como experimento y se mantiene como alternativa futura.

En conjunto, la literatura sugiere tres desafíos recurrentes: tamaño aparente reducido, contexto de la escena y control de falsas alarmas. Este trabajo se concentra experimentalmente en el primero mediante análisis geométrico y entrenamiento Multi-Scale, reservando la integración temporal y contextual para el prototipo práctico.

III. Metodología

A. Datos y preparación

Se utilizó YouTube Gun Detection Dataset (YouTube-GDD), compuesto por frames procedentes de videos y anotaciones YOLO para *person* (clase 0) y *gun* (clase 1). La copia descargada contenía inicialmente 5 000 imágenes. La auditoría verificó correspondencia imagen–etiqueta, estructura de líneas, identificadores de clase y coordenadas normalizadas. Se identificó un frame visualmente positivo sin anotación válida; para evitar introducir una etiqueta manual subjetiva, fue excluido mediante un procedimiento reproducible y se conservó intacta la fuente original. La Tabla I muestra la distribución final del conjunto de datos.

La distribución conserva 4 523 instancias de *person* y 8 032 de *gun*, para un total de 12 555 cajas. Test permaneció reservado durante el desarrollo de B1, M1 y M2. La selección se realizó únicamente con Validation y el checkpoint ganador fue congelado antes de abrir Test.

TABLE I
Distribución del conjunto de datos

Split	Imágenes	person	gun
Train	3,997	3,713	6,432
Validation	494	410	712
Test	508	400	888

Fuente: Elaboración propia.

B. Fundamento de redes neuronales convolucionales

Una CNN aprende filtros convolucionales capaces de responder a patrones locales de la imagen. Para una entrada X y un kernel K, la convolución puede expresarse de forma simplificada como:

$$Y(i,j) = \sum_m \sum_n X(i+m, j+n) K(m,n) + b. \quad (1)$$

Durante el *forward pass* se calculan predicciones; la función de pérdida cuantifica el error y *backpropagation* propaga gradientes para actualizar los pesos mediante el optimizador.

En detección de objetos, la red predice tanto la clase como los parámetros geométricos de una caja. La calidad de localización se evalúa mediante Intersection over Union (IoU):

$$\mathrm{IoU}(B_p, B_{gt}) = \frac{|B_p \cap B_{gt}|}{|B_p \cup B_{gt}|}, \quad (2)$$

donde B_p es la caja predicha y B_{gt} la caja de referencia.

1) Aprendizaje supervisado y propagación hacia adelante: El entrenamiento se plantea como aprendizaje supervisado: cada imagen de Train se presenta junto con cajas delimitadoras y etiquetas de clase. En las capas tempranas predominan respuestas a bordes, contrastes y texturas; en capas más profundas estas respuestas se combinan en patrones de mayor nivel. El proceso se repite durante 100 epochs para B1, M1 y M2. Mantener el mismo optimizador, semilla, inicialización y número de epochs busca que las diferencias puedan atribuirse a la variable experimental de cada modificación. Como una sola semilla no estima la variabilidad entre entrenamientos, B1 y M2 se repitieron con semillas adicionales (Sección IV-G).

2) Convolución, escalas y objetos pequeños: Una convolución aplica un kernel aprendido sobre regiones locales de la imagen y produce mapas de características. El campo receptivo aumenta conforme se profundiza en la red, permitiendo integrar contexto espacial. Sin embargo, las operaciones de reducción espacial necesarias para construir representaciones semánticas pueden dificultar la conservación de información de objetos muy pequeños. Por esta razón, los detectores modernos combinan características de distintas resoluciones.

3) Bounding boxes, IoU y supresión de duplicados: Cada detección produce una caja delimitadora, una confianza y una clase. Para determinar si una predicción coincide con un objeto real se utiliza Intersection over Union (IoU), definida

como el área de intersección entre la caja predicha y la Ground Truth dividida entre el área de su unión. En el análisis diagnóstico se fijó matching IoU = 0.50. Durante inferencia, Non-Maximum Suppression elimina duplicados conservando las cajas de mayor confianza. Para mantener comparabilidad entre B1, M1 y M2, el diagnóstico utilizó confidence = 0.001 y NMS IoU = 0.60.

C. Arquitectura YOLOv5s

YOLOv5s [5] es un detector de una etapa diseñado para producir predicciones directamente a partir de la imagen. Conceptualmente puede dividirse en backbone, neck y detection head. El backbone transforma la imagen en mapas de características jerárquicos. El neck combina información procedente de diferentes niveles para conservar simultáneamente semántica de alto nivel y detalle espacial. El detection head genera cajas, objectness y probabilidades de clase en múltiples escalas.

La detección en tres escalas (*strides* 8, 16 y 32) es especialmente relevante en este proyecto porque las armas presentan tamaños muy heterogéneos. Esta propiedad de la arquitectura es distinta del entrenamiento Multi-Scale de M2, que solo varía la resolución de las imágenes de entrada. Los anchors proporcionan formas iniciales de referencia para la regresión de cajas. Sin embargo, el diagnóstico posterior mostró que los hard gun no tenían peor compatibilidad geométrica con los anchors que los objetos detectados, por lo que no se justificó modificar los anchors como siguiente intervención experimental.

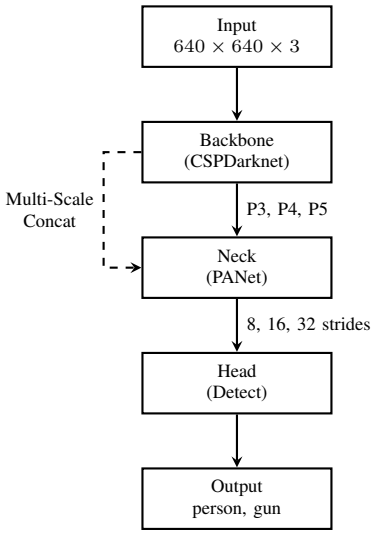


Fig. 1. Arquitectura de YOLOv5s utilizada en B1, M1 y M2. La red no se modificó; el entrenamiento Multi-Scale de M2 cambia la resolución de entrada por lote.

D. Métricas

Para cada clase se contabilizan verdaderos positivos (TP), falsos positivos (FP) y falsos negativos (FN). Las métricas se definen como:

$$\text{Precision} = \frac{TP}{TP + FP}, \tag{3}$$

$$\text{Recall} = \frac{TP}{TP + FN}. \tag{4}$$

AP resume el compromiso Precision–Recall y mAP promedia AP entre clases. Se reportan mAP@0.5 y mAP@0.5:0.95. Para el análisis diagnóstico por objeto se utilizó matching IoU$\geq$0.50, confidence 0.001 y NMS IoU 0.60, de forma consistente entre B1, M1 y M2. Para comparar B1 y M2 entre semillas se reportan la media y la desviación estándar (DE) de cada métrica y una prueba de permutación exacta y bilateral sobre la diferencia de medias.

E. Diseño experimental controlado

Se utilizó YOLOv5 v7.0, commit 915bbf2, pesos iniciales yolov5s.pt preentrenados en COCO [8], hiperparámetros por defecto, 100 epochs, batch 16, 2 workers, SGD y seed 0. Las tres configuraciones utilizaron el mismo dataset. En M2, la opción `--multi-scale` elige al azar, en cada lote, una resolución entre 320 y 960 píxeles en múltiplos de 32; la evaluación se realiza a 640. Para estimar la variabilidad entre entrenamientos, B1 y M2 se repitieron con las semillas 0, 1 y 2 en una NVIDIA GeForce RTX 4090 con el mismo protocolo. M1 no se replicó, porque su diferencia con B1 es muy superior a esa variabilidad. La estrategia fue modificar una variable experimental principal por etapa, como se muestra en la Tabla II.

TABLE II
CONFIGURACIONES EXPERIMENTALES

Exp.	Modelo	Input	Multi-Scale	Hipótesis
B1	YOLOv5s	640	No	Línea base
M1	YOLOv5s	1280	No	Mayor resolución favorece gun pequeñas
M2	YOLOv5s	640	Sí	Variación de escala mejora robustez

Fuente: Elaboración propia.

B1 establece el punto de referencia. M1 surge del hallazgo de falsos negativos pequeños y prueba si duplicar la resolución conserva información útil. Tras observar que M1 no mejora, se realiza un análisis más profundo de los errores. M2 incorpora Multi-Scale manteniendo la resolución base de 640 y el resto de condiciones principales, buscando exponer al detector a variaciones de escala durante el entrenamiento.

1) Principio de ablación controlada: El diseño experimental siguió una lógica de ablación: en cada propuesta se modificó una variable principal mientras se conservaron las demás condiciones relevantes. B1 fijó la referencia. M1 cambió únicamente la resolución de entrada de 640 a 1280. Tras comprobar que M1 empeoraba tanto las métricas globales como los falsos negativos de gun, el análisis dejó de centrarse en "más píxeles" y pasó a caracterizar los errores. M2 volvió a 640 y activó Multi-Scale durante entrenamiento.

F. Protocolo de selección y congelamiento

La selección se realizó exclusivamente con Validation. M2 obtuvo el mayor fitness y redujo los FN de gun frente a B1. Antes de acceder a Test, su best.pt fue congelado y registrado mediante SHA256 `084fbe295262885fc116ebc3a4dff110040ab2f95c56b211c570a1cae6f1a8fa`. Test se utilizó después una sola vez para evaluación final y sus resultados no originaron nuevos ajustes. Las réplicas con otras semillas se ejecutaron después, solo sobre Validation, y no modificaron la selección.

IV. Resultados

A. Línea base B1

B1 completó 100 épocas en 1.823 h sobre una NVIDIA A100-SXM4-40GB. El mejor epoch según fitness fue 84. En Validation obtuvo Precision 0.928400, Recall 0.798420, mAP@0.5 0.851590, mAP@0.5:0.95 0.669360 y fitness 0.687583. Bajo el protocolo diagnóstico, los 712 objetos *gun* de Validation produjeron 580 TP y 132 FN. La mediana de área normalizada fue 0.178150 para los TP y 0.005678 para los FN, lo que mostró que los objetos no detectados eran mucho más pequeños.

B. M1: incremento de resolución

M1 mantuvo arquitectura, pesos iniciales, optimizador, épocas, batch, workers, seed y dataset, modificando únicamente la resolución de 640×640 a 1280×1280. Completó 100 épocas en 3.206 h. El mejor epoch fue 99, con Precision 0.779180, Recall 0.764080, mAP@0.5 0.790320, mAP@0.5:0.95 0.524020 y fitness 0.550650.

La comparación completa de los tres experimentos se presenta en la Tabla IV. La hipótesis de M1 no fue respaldada. M1 aumentó el costo de entrenamiento aproximadamente 75.9% respecto de B1. En *gun*, los TP disminuyeron de 580 a 565 y los FN aumentaron de 132 a 147.

C. Evolución de pérdidas M1

M1 sí mostró aprendizaje en Train: *box_loss* disminuyó 77.44%, *obj_loss* 59.77% y *cls_loss* 93.41%, *val/obj_loss* aumentó 8.37%. Una reducción de pérdidas de entrenamiento no implica necesariamente mejor generalización.

D. Análisis cruzado B1 frente a M1

Se reconstruyó el estado de los mismos 712 objetos *gun* bajo ambos modelos. De ellos, 543 (76.26%) fueron detectados por ambos, 110 (15.45%) fallados por ambos, 22 recuperados por M1 y 37 perdidos por M1. El balance neto de M1 fue de 15 detecciones menos. Los 110 objetos fallados por ambos presentaron mediana de área normalizada 0.004405, frente a 0.205509 para los detectados por ambos; el cociente de medianas fue 0.021437.

El 80.00% de los objetos fallados por ambos modelos se encontraba en regiones periféricas, frente a 19.52% de los detectados por ambos. Además, 102 de los 110 objetos difíciles aparecían en imágenes con más de un *gun*. Dos frames consecutivos de la secuencia `O36UzaP9O1c_30_60` concentraron 60 de los 110 errores persistentes (54.55%). En conjunto contenían 87 armas; 82 pertenecían al primer cuartil de tamaño y 76 eran periféricas. La tasa de error persistente en estas escenas fue 68.97%, frente a 8.00% para los restantes 625 objetos de Validation. La Fig. 2 muestra armas pequeñas que M2 tampoco detectó: armas exhibidas en el borde de un estante, armas parcialmente cubiertas por el cuerpo o lejanas, e incluso un icono de arma en la interfaz de un videojuego, este último más cercano a un caso límite de anotación que a un error del detector.

TABLE III
Errores persistentes por cuartil de área en Validation

Cuartil	Total gun	Fallado ambos	Tasa
Q1, más pequeños	178	94	52.81%
Q2	178	15	8.43%
Q3	178	1	0.56%
Q4, más grandes	178	0	0.00%

Fuente: Elaboración propia.

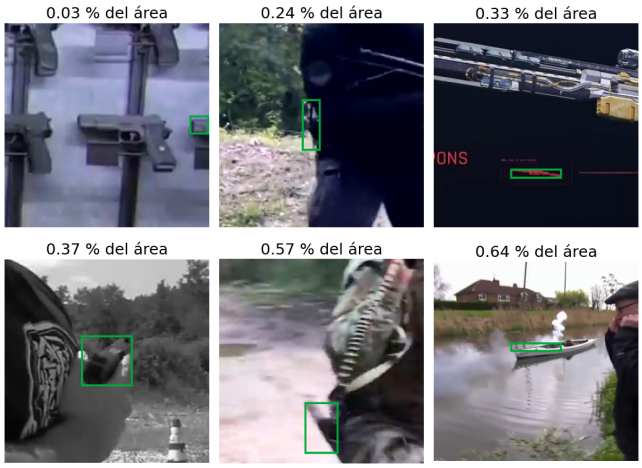


Fig. 2. Armas pequeñas de Validation que M2 no detectó (sin ninguna predicción solapada a confianza 0.25). Recortes ampliados de seis videos distintos; en verde, la anotación. Sobre cada recorte, el área de la caja respecto de la imagen.

E. Diagnóstico de anchors y representatividad

Los nueve anchors de YOLOv5s fueron auditados frente a Train y Validation. Para los 712 objetos *gun* de Validation, la mediana del mejor anchor IoU fue 0.591511; en el grupo detectado por ambos modelos fue 0.573575 y en el grupo fallado por ambos 0.607536. Los objetos fallados no presentaron peor compatibilidad geométrica con los anchors, por lo que no se justificó modificarlos sin evidencia adicional.

Train contenía 6 432 objetos *gun*. El 31.79% se encontraba dentro del rango Q1 definido por Validation. Se definió un patrón difícil como área menor o igual al percentil 25 de Validation, posición periférica e imagen con más de un arma. Este patrón apareció en 1 559 objetos de Train (24.24%), 130

de Validation (18.26%) y 80 de los 110 objetos fallados por ambos (72.73%). El 80.97% de las imágenes de Train con armas contenía un solo objeto, aunque existían escenas de alta densidad, con un máximo de 56 armas.

F. M2: entrenamiento Multi-Scale

A partir de estos resultados se descartó continuar aumentando resolución y no se encontró evidencia suficiente para modificar anchors. Se eligió Multi-Scale porque introduce variación de escala durante el entrenamiento manteniendo la arquitectura YOLOv5s y la resolución nominal de evaluación. M2 mantuvo entrada 640×640, 100 épocas, batch 16, dos workers, SGD, seed 0, mismo dataset, pesos iniciales e hiperparámetros de B1. La única variable experimental principal fue Multi-Scale=True. Completó 100 épocas en 1.837 h.

TABLE IV
COMPARACIÓN DE CONFIGURACIONES EN VALIDATION

Métrica	**B1**	**M1**	**M2**
Precision	0.928400	0.779180	0.912800
Recall	0.798420	0.764080	0.812900
mAP@0.5	0.851590	0.790320	0.861920
mAP@0.5:0.95	0.669360	0.524020	0.680160
Fitness	0.687583	0.550650	0.698336
Gun TP	580	565	590
Gun FN	132	147	122
Tiempo (h)	1.823	3.206	1.837

Fuente: Elaboración propia.

En esta corrida, M2 obtuvo el mayor fitness. Frente a B1, el fitness aumentó 0.010753 (1.56%), mAP@0.5 aumentó 0.010330 y mAP@0.5:0.95 aumentó 0.010800. Recall mejoró de 0.798420 a 0.812900. En *gun*, los TP aumentaron de 580 a 590 y los FN disminuyeron de 132 a 122, una reducción de 7.58% en el protocolo diagnóstico de bajo umbral.

De los 110 objetos fallados simultáneamente por B1 y M1, M2 recuperó 15 (13.64%) y 95 continuaron fallados. Los recuperados presentaron mediana de área 0.008043, mientras que los todavía fallados tuvieron 0.004063. La mediana del grupo recuperado fue aproximadamente 1.98 veces la del grupo residual. M2 alcanzó su máximo fitness en el epoch 98. En las últimas diez épocas, Precision promedio fue 0.910977, Recall 0.809698, mAP@0.5 0.861233 y mAP@0.5:0.95 0.676529. Las pérdidas de validación finalizaron en 0.033753 para box, 0.019024 para objectness y 0.003249 para clasificación en el epoch seleccionado. La Fig. 3 muestra que las curvas de B1 y M2 se superponen durante todo el entrenamiento, mientras M1 queda por debajo desde las primeras épocas; la Fig. 4 muestra lo mismo en la curva Precision–Recall de *gun*. La Sección IV-G examina si la ventaja de M2 se mantiene con otras semillas. La Sección IV-G examina si esta ventaja se mantiene con otras semillas.

G. Variabilidad entre semillas

Las comparaciones anteriores corresponden a un único entrenamiento por configuración. En YOLOv5, la semilla



Fig. 3. Evolución del mAP@0.5:0.95 de Validation por época para B1, M1 y M2 (corrida inicial, semilla 0).

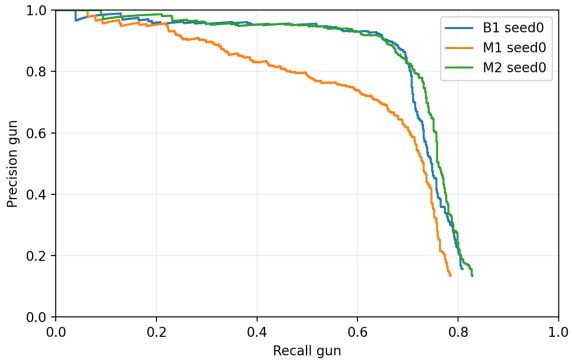


Fig. 4. Curva Precision–Recall de la clase *gun* en Validation para B1, M1 y M2 (semilla 0), con el evaluador complementario.

determina la inicialización de la cabeza, el orden de los lotes y el aumento de datos, y los cálculos en GPU no son completamente deterministas; por ello, dos entrenamientos con la misma configuración pueden diferir en una magnitud comparable a la de una intervención [14]. Para estimar esa variación, B1 y M2 se repitieron con las semillas 0, 1 y 2 en una NVIDIA GeForce RTX 4090. Junto con la corrida inicial en la A100, cada configuración suma cuatro entrenamientos. Todos los checkpoints se reevaluaron en el mismo equipo y con el mismo procedimiento (Tabla V y Fig. 5).

Con cuatro entrenamientos por configuración no se observa una diferencia medible entre B1 y M2: el mAP@0.5:0.95 medio fue 0.679 ± 0.007 para B1 y 0.674 ± 0.005 para M2 ($p = 0.31$), y las omisiones de *gun* a confianza 0.25 fueron prácticamente iguales (211.0 frente a 210.8). El recall de armas pequeñas, que motivó M2, tampoco cambió de forma apreciable (0.167 frente a 0.180; $p = 0.51$). Las cuatro corridas de B1 abarcan de 0.670 a 0.688 en mAP@0.5:0.95, un rango mayor que la ventaja de 0.011 que M2 mostró con una sola semilla. Además, con la misma semilla 0, cambiar de la A100 a la RTX 4090 movió B1 de 0.670 a 0.678 y M2 de 0.680 a 0.671, invirtiendo el orden entre ambos. La prueba de permutación considera las 70 formas de repartir las ocho corridas entre dos grupos, de modo que el menor valor

TABLE V
VARIABILIDAD ENTRE SEMILLAS DE B1 Y M2 EN VALIDATION

Exp.	Sem. / GPU	mAP50	mAP50:95	gun 50:95	gun FN	gun peq.
B1	0 / A100	0.852	0.670	0.518	215	0.143
B1	0 / RTX 4090	0.856	0.678	0.524	214	0.165
B1	1 / RTX 4090	0.863	0.688	0.539	209	0.165
B1	2 / RTX 4090	0.853	0.680	0.518	206	0.195
B1: media ± DE		0.856±0.005	0.679±0.007	0.525±0.010	211.0±4.2	0.167±0.022
M2	0 / A100	0.861	0.680	0.529	214	0.195
M2	0 / RTX 4090	0.861	0.671	0.515	217	0.143
M2	1 / RTX 4090	0.862	0.676	0.524	212	0.173
M2	2 / RTX 4090	0.853	0.669	0.519	200	0.211
M2: media ± DE		0.859±0.004	0.674±0.005	0.522±0.006	210.8±7.5	0.180±0.029
M2 − B1		+0.003	-0.005	-0.003	-0.2	+0.013
p (permutación)		0.46	0.31	0.71	1.00	0.51

mAP con `val.py`; gun FN y gun peq. (recall de armas con área ≤1% de la imagen) con el evaluador complementario a confianza 0.25. Fuente: elaboración propia.

p posible es 0.029. En cambio, la caída de M1 respecto de B1 en mAP@0.5:0.95 (0.145) equivale a unas veinte desviaciones estándar entre semillas, por lo que ese resultado negativo es robusto aun con una sola corrida.

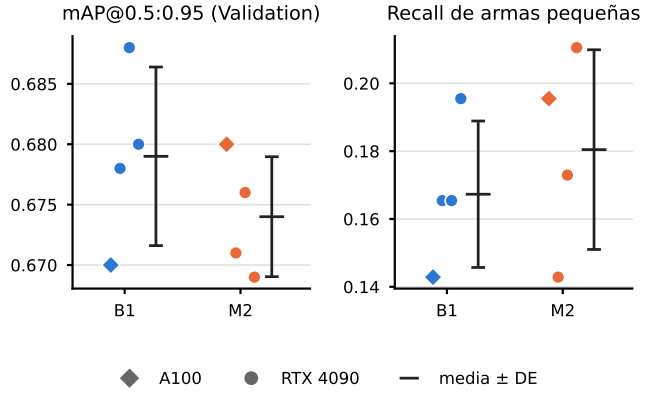


Fig. 5. Variabilidad entre entrenamientos de B1 y M2. Cada punto es un entrenamiento (rombo: A100; círculo: RTX 4090); la barra indica media ± DE. Las nubes de ambas configuraciones se solapan en las dos métricas.

H. Evaluación independiente de M2 sobre Test

El checkpoint M2, seleccionado solo mediante Validation y congelado antes de acceder a Test, se evaluó mediante `val.py` sobre 508 imágenes con 1 288 objetos (400 *person* y 888 *gun*). B1 y M1 no se evaluaron en Test dentro del protocolo reportado.

TABLE VI
EVALUACIÓN FINAL INDEPENDIENTE DE M2 EN TEST

Clase	Inst.	Prec.	Recall	mAP50	mAP50:95
Global	1 288	0.875	0.780	0.845	0.659
person	400	0.933	0.863	0.926	0.812
gun	888	0.817	0.697	0.765	0.506

Respecto a Validation, el mAP@0.5 disminuyó 0.016920 y el mAP@0.5:0.95 disminuyó 0.021160. La clase *gun* conservó mayores dificultades: su Recall en Test fue 0.697 frente a 0.863 para *person*.

I. Diagnóstico de falsos positivos

El análisis diagnóstico sobre Validation utilizó `confidence=0.001`, umbral distinto al de operación del MVP. Para *gun*, B1 registró 3 232 FP y M2 3 978 FP, aunque M2 redujo los FN de 132 a 122. Estos conteos corresponden a la inspección de predicciones con umbral muy bajo y no deben confundirse con la Precision oficial de `val.py` ni con la tasa de falsas alarmas temporales en CCTV. Al umbral de 0.25, la mayoría de los FP de *gun* de M2 no son falsas alarmas sobre escenas sin armas: de 104, 75 se solapan con un arma anotada pero con IoU menor que 0.50, 10 son duplicados y solo 19 no se solapan con ninguna arma. La Fig. 6 muestra los cuatro FP de M2 con mayor confianza: en tres de ellos la predicción cubre el arma mientras la anotación incluye también la mano o parte del brazo, lo que apunta a criterios de anotación poco homogéneos en YouTube-GDD. De las 214 armas omitidas, 121 tienen una predicción cercana con IoU menor que 0.50 y 93 no tienen ningún candidato.



Fig. 6. Los cuatro falsos positivos de *gun* de M2 con mayor confianza en Validation (videos distintos). Verde: anotación; rojo: predicción. Todos se deben a un IoU menor que 0.50 con un arma anotada, no a escenas sin armas.

J. Prueba de concepto: motor temporal V3–V5

Para evaluar la aplicabilidad del detector M2 sobre video, se desarrolló un MVP denominado *AI Security Monitoring System*, con interfaz Streamlit, inferencia espacial y filtrado temporal. Se compararon ajustes sucesivos del motor temporal: V3 incorpora persistencia y memoria frente a oclusiones, V4 añade seguimiento y movimiento y V5 incorpora restricciones por fondo estático. En el video de referencia de 2 836 frames, los conteos y tiempos registrados se resumen en la Tabla VII.

V3 conservó más confirmaciones del motor y mayor velocidad en esta prueba. Los conteos de eventos no constituyen por sí solos verdaderos positivos ni prueban una reducción cuantificada de falsos positivos: esa comparación requiere anotaciones temporales independientes. Se seleccionó V3 como configuración operativa del MVP para esta demostración; la selección no altera el checkpoint M2 ni sus métricas sobre Test.

TABLE VII
COMPARACIÓN OPERATIVA DE MOTORES TEMPORALES EN UN VIDEO CCTV

Indicador	V3	V4	V5
Ciclos de inferencia	473	473	473
Ciclos con *gun*	66	66	66
Confirmaciones del motor	66	14	1
Activaciones memoria/oclusión	41	10	0
Alertas críticas	25	4	1
Eventos contabilizados	7	3	1
Tiempo de proceso (s)	259.86	381.41	506.73
Velocidad (fps)	10.91	7.44	5.60

V. DISCUSIÓN

La comparación controlada muestra que aumentar de 640 a 1280 la resolución nominal de entrada no garantiza mejoras en detección. M1 empleó 75.9% más tiempo de entrenamiento que B1 y redujo fitness y mAP; además, incrementó los falsos negativos de *gun*. El análisis por objeto mostró que el error persistente se concentraba en objetos pequeños, periféricos y escenas con múltiples armas. La compatibilidad geométrica de anchors no explicó por sí sola esos fallos, por lo que se evaluó entrenamiento Multi-Scale como alternativa.

En la corrida inicial, M2 obtuvo la mejor combinación de métricas oficiales de Validation, con una mejora modesta del fitness (0.687583 a 0.698336) y una reducción diagnóstica de FN de *gun* (132 a 122). Las réplicas no sostienen esa lectura: con cuatro entrenamientos por configuración, la diferencia media de mAP@0.5:0.95 fue -0.005 ($p = 0.31$), las omisiones de *gun* a confianza 0.25 fueron prácticamente iguales y el recall de armas pequeñas se mantuvo entre 0.17 y 0.18 en ambos. La ventaja inicial de M2 es del mismo orden que la variación entre semillas y que el cambio producido solo por entrenar en otra GPU. Por tanto, el entrenamiento Multi-Scale, con la configuración por defecto de YOLOv5, no resolvió la dificultad de las armas pequeñas. El resultado negativo de M1, en cambio, es robusto. Por otra parte, en el diagnóstico a `confidence=0.001` los FP de *gun* aumentaron de 3 232 a 3 978. El resultado evidencia un compromiso entre sensibilidad y activaciones espurias que no se aprecia únicamente con mAP. Esos FP a bajo umbral no deben compararse directamente con la Precision oficial calculada por `val.py` ni con las alertas del MVP, que emplea criterios adicionales.

A. Implementación del sistema de monitoreo (MVP)

Se desarrolló un MVP en Streamlit, *AI Security Monitoring System*, para procesar imágenes y videos MP4 con el checkpoint M2 congelado. El detector espacial V3 aplica inferencia en imagen completa a ambas clases y procesamiento adicional por regiones para *person*; el motor de video incorpora persistencia, asociación espacial mediante IoU y memoria temporal para reducir la discontinuidad producida por oclusiones. El sistema permite visualizar cajas y resultados, generar video procesado y registrar eventos en CSV. La salida `WEAPON_CONFIRMED` indica confirmación de una regla temporal, no certificación independiente de que existe un arma real.

En un video de CCTV de 2 836 frames, V3 procesó la secuencia en 259.86 s (10.91 fps), frente a 381.41 s (7.44 fps) de V4 y 506.73 s (5.60 fps) de V5. V3 registró siete eventos, frente a tres y uno, respectivamente. La comparación sugiere que los filtros más restrictivos pueden suprimir o retrasar señales relevantes. Sin *ground truth* temporal exhaustivo, no corresponde interpretar estos conteos como sensibilidad, especificidad o tasas validadas de falsos positivos; la evidencia sustenta una demostración funcional, no una validación de despliegue.

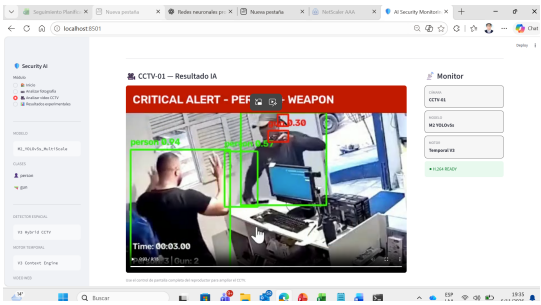


Fig. 7. Interfaz del MVP en Streamlit con video procesado y línea temporal de eventos.

B. Costo computacional y reproducibilidad

Los logs reportan 1.823 h, 3.206 h y 1.837 h de entrenamiento para B1, M1 y M2, respectivamente, en una NVIDIA A100-SXM4-40GB (6.866 h). Las seis réplicas de B1 y M2 sumaron 8.02 h en una NVIDIA GeForce RTX 4090, para un total de 14.89 horas de GPU de entrenamiento, sin contar validaciones ni pruebas de inferencia. La implementación distribuida del MVP, en cambio, está configurada para CPU, la implementación distribuida del MVP está configurada para CPU; por tanto, el FPS de video no es comparable directamente con el costo del entrenamiento. Se documentaron versiones, parámetros, semillas, pesos iniciales, archivos `results.csv` y checkpoints. El hash SHA256 del checkpoint seleccionado permite comprobar su identidad entre la evaluación y el MVP.

La reproducibilidad se apoya en particiones separadas por secuencia, configuraciones explícitas y selección exclusiva mediante Validation. El repositorio organiza documentación, scripts, configuraciones y resultados; para una entrega plenamente verificable debe mantenerse accesible el enlace definitivo al checkpoint, junto con las instrucciones exactas de reproducción. Las diferencias de dependencias entre el entorno de entrenamiento y la aplicación local deben quedar documentadas. Las réplicas muestran además un límite de la reproducibilidad: con la misma semilla 0, B1 obtuvo 0.670 en la A100 y 0.678 en la RTX 4090, y M2, 0.680 y 0.671. Fijar la semilla no garantiza el mismo resultado entre GPU y versiones de bibliotecas distintas; por eso las diferencias entre configuraciones deben evaluarse con varias semillas [14].

C. Limitaciones, amenazas a la validez y uso responsable

La variabilidad entre semillas se estimó con cuatro entrenamientos de B1 y de M2, dos de ellos con la misma semilla en hardware distinto; M1 se entrenó una sola vez. Con tan pocas corridas, la prueba solo detectaría diferencias grandes: una diferencia menor que unas 0.01 en mAP@0.5:0.95 no puede confirmarse ni descartarse.. Aunque las particiones se separaron por video para reducir fugas de información, el conjunto YouTube-GDD no representa necesariamente todas las cámaras, escenarios, armas, condiciones de iluminación ni técnicas de compresión usadas en CCTV. En Test el Recall de *gun* fue 0.697, lo que muestra que persisten omisiones relevantes.

La prueba del MVP se limitó a un video de referencia y no constituye validación en producción. Las reglas de persistencia pueden atenuar activaciones aisladas, pero también retrasar o suprimir detecciones reales. Las alertas, incluso cuando se etiquetan como críticas, no permiten atribuir identidad, intención, posesión de un arma o culpabilidad. El sistema se plantea exclusivamente como apoyo a la revisión humana y requeriría antes de un despliegue evaluaciones externas, tratamiento adecuado de datos personales, controles de acceso y políticas de conservación de imágenes y registros.

D. Trabajo futuro

Se propone ampliar y etiquetar material CCTV con escenas negativas, oclusiones, variaciones nocturnas y armas pequeñas, así como incorporar falsos positivos reales como ejemplos difíciles de entrenamiento. También se evaluarán estrategias de recorte o *tiling* específico para *gun*, superresolución y diseños de detección para objetos pequeños. Estas alternativas no formaron parte de los experimentos B1–M2 reportados, y cualquier nueva intervención debería compararse con varias semillas desde el inicio. Para el motor temporal se requiere un conjunto con anotaciones por evento que permita medir precisión, recall, falsas alarmas por hora, latencia y estabilidad entre cámaras.

VI. Conclusiones

Se estableció una línea base YOLOv5s reproducible y se caracterizó cuantitativamente su principal dificultad: la detección de armas pequeñas. M1 demostró que duplicar la resolución a 1280 no mejora necesariamente el desempeño y aumenta significativamente el costo; su caída equivale a unas veinte desviaciones estándar entre semillas. El análisis sistemático mostró que los errores persistentes se concentran en objetos pequeños, periféricos y escenas densas. M2, basado en entrenamiento Multi-Scale, obtuvo el mejor fitness de Validation en la corrida inicial (0.698336), pero al reunir cuatro entrenamientos por configuración su diferencia con B1 desapareció (mAP@0.5:0.95 de 0.674 frente a 0.679; $p = 0.31$) y el recall de armas pequeñas siguió por debajo de 0.20. Una sola semilla no basta para atribuir a una intervención diferencias del orden de 0.01 en mAP.

La evaluación independiente de M2 obtuvo mAP@0.5 de 0.845 y mAP@0.5:0.95 de 0.659. Sin embargo, la dificultad residual de la clase *gun*, cuyo Recall fue 0.697, justifica mantener la revisión humana y motiva futuras mejoras orientadas a objetos extremadamente pequeños, evaluadas con varias semillas. Como trabajo futuro se propone validar el prototipo de video con anotaciones temporales, medir su rendimiento en el hardware de despliegue y evaluar métodos adicionales para mejorar la detección sin comprometer la reproducibilidad ni el uso responsable del sistema.

Contribuciones

TABLE VIII
Contribuciones del Equipo

Integrante	Tareas en las que participó
Jose Miguel Alvarado Moreno	Integración del repositorio y automatización del flujo (auditoría de datos, entrenamiento, validación y exportación); reevaluación de B1, M1 y M2 en RTX 4090; réplicas de B1 y M2 con tres semillas y análisis de variabilidad; análisis de errores, umbrales y rendimiento; pruebas y documentación.
Dina Yesica Huamán Huallpa	Planteamiento del problema; preparación del dataset, notebook y entrenamiento de B1 junto con Jose Gabriel Laymito Quispe; análisis de errores y preparación de M1; documentación del código; redacción del artículo y diapositivas.
Jose Gabriel Laymito Quispe	Planteamiento e implementación del notebook experimental; preparación del dataset; entrenamiento de B1, M1 y M2; análisis de errores y evaluación final en Test.
Christopher Nuñez	Entrenamiento, redacción del artículo y diapositivas.
Frank Huaman Cancho	Interpretación de resultados; revisión y mejora del artículo; revisión del código Python y de la estructura del pipeline; diapositivas.

Fuente: Elaboración propia.

Disponibilidad de Código y Uso de Herramientas de IA

Se utilizó ChatGPT/Claude para la revisión de redacción y depuración de scripts en Python, pero todos los resultados y experimentos son propios

References

[1] R. Olmos, S. Tabik, and F. Herrera, "Automatic handgun detection alarm in videos using deep learning," *Neurocomputing*, vol. 275, pp. 66–72, 2018.

[2] J. L. Salazar González, C. Zaccaro, J. A. Álvarez-García, L. M. Soria Morillo, and F. Sancho Caparrini, "Real-time gun detection in cctv: An open problem," *Neural Networks*, vol. 132, pp. 297–308, 2020.

[3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[4] J. Redmon and A. Farhadi, "YOLO9000: Better, faster, stronger," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 7263–7271.

[5] G. Jocher *et al.*, "ultralytics/yolov5: v7.0 – YOLOv5 SOTA realtime instance segmentation," Zenodo, 2022.

[6] C.-Y. Wang, H.-Y. M. Liao, Y.-H. Wu, P.-Y. Chen, J.-W. Hsieh, and I.-H. Yeh, "CSPNet: A new backbone that can enhance learning capability of CNN," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2020, pp. 390–391.

[7] S. Liu, L. Qi, H. Qin, J. Shi, and J. Jia, "Path aggregation network for instance segmentation," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 8759–8768.

[8] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *Computer Vision – ECCV 2014*, ser. Lecture Notes in Computer Science, vol. 8693, 2014, pp. 740–755.

[9] Y. Gu, X. Liao, and X. Qin, "YouTube-GDD: A challenging gun detection dataset with rich contextual information," arXiv:2203.04129, 2022.

[10] A. Velasco-Mata, J. Ruiz-Santaquiteria, N. Vallez, and O. Deniz, "Using human pose information for handgun detection," *Neural Computing and Applications*, vol. 33, pp. 17 273–17 286, 2021.

[11] D. Berardini, L. Migliorelli, A. Galdelli, E. Frontoni, A. Mancini, and S. Moccia, "A deep-learning framework running on edge devices for handgun and knife detection from indoor video-surveillance cameras," *Multimedia Tools and Applications*, vol. 83, pp. 19 109–19 127, 2024.

[12] P. Corral-Sanz, A. Barreiro-Garrido, A. B. Moreno, and A. Sanchez, "On the influence of artificially distorted images in firearm detection performance using deep learning," *PeerJ Computer Science*, vol. 10, p. e2381, 2024.

[13] D. Berardini, L. Migliorelli, A. Galdelli, and M. J. Marín-Jiménez, "Edge artificial intelligence and super-resolution for enhanced weapon detection in video surveillance," *Engineering Applications of Artificial Intelligence*, vol. 140, p. 109684, 2025.

[14] X. Bouthillier, P. Delaunay, M. Bronzi *et al.*, "Accounting for variance in machine learning benchmarks," in *Proceedings of Machine Learning and Systems (MLSys)*, vol. 3, 2021, pp. 747–769.